CSF205:
Web Service Development

Week 4: RESTful Desing and Implementation



Swansea University
Prifysgol Abertawe

Department of Computer Science
Applied Software Engineering

Learning Outcomes



Lecture

- Understanding the REST principles and best practices
- Design choices and the challenges of flexibility
- Code-First approach to web service implementation
- RESTful web services with Spring Boot



Lab

- Designing your first set of REST endpoints
- Implementing and deploying your own web service with Spring Boot.

PART 1

RESTful Principles

RESTful Architectural Style

Roy Fielding's Dissertation on RESTful Architectural Style [1].

Basis for REST APIs.

6 Constraints:

a) Uniform interface

b) Stateless

c) Cacheable

d) Client-Server

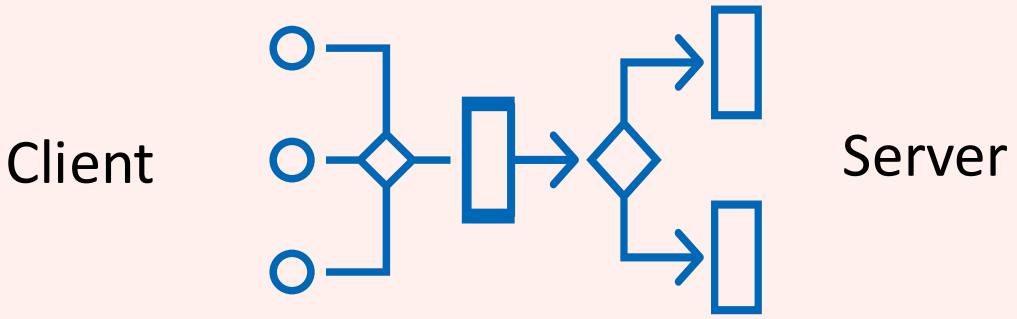
e) Layered System

f) Code on Demand

[1] http://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

REST: Uniform Interface

Defines the interface between clients and servers.



Simplifies and decouples the architecture.

Dictated by four guiding principles of the uniform interface.

REST: Guidelines for Uniform Interfaces

Resource-based:

URIs identify individual resources.

Manipulation of Resources through Representations:

Methods and Representations can change resources.

Self-descriptive Messages:

Each message should describe how it should be processed.

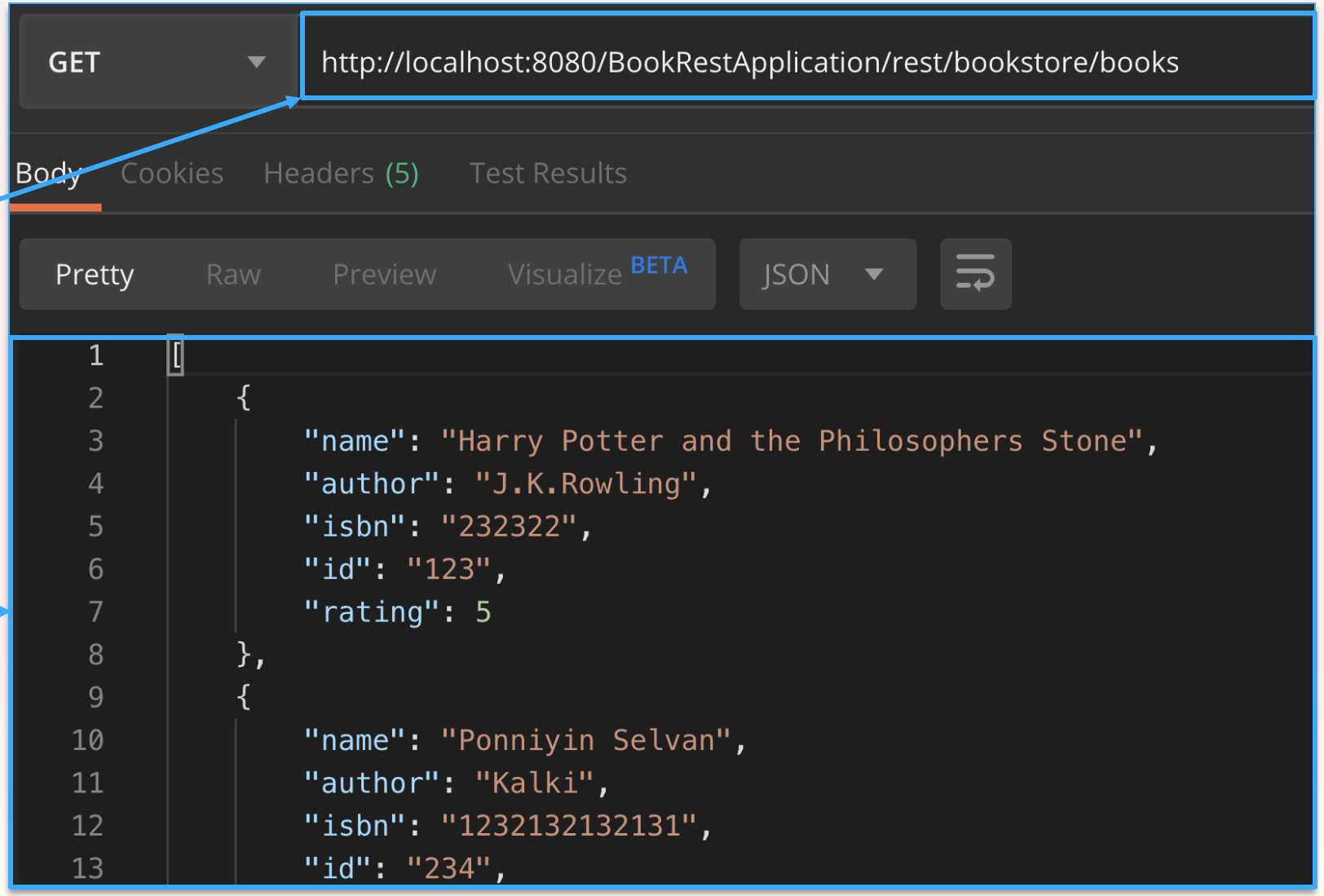
(HATEOS):

Hypermedia **a**s **t**he **E**ngine **o**f Application **S**tate.
Links to related resources are included in messages.

REST: Resource vs. Representation

Resource
URI.

Representation:
Plaintext
JSON list



JSON: Specifications

Introduction to components of the language:

- https://www.json.org/json-en.html



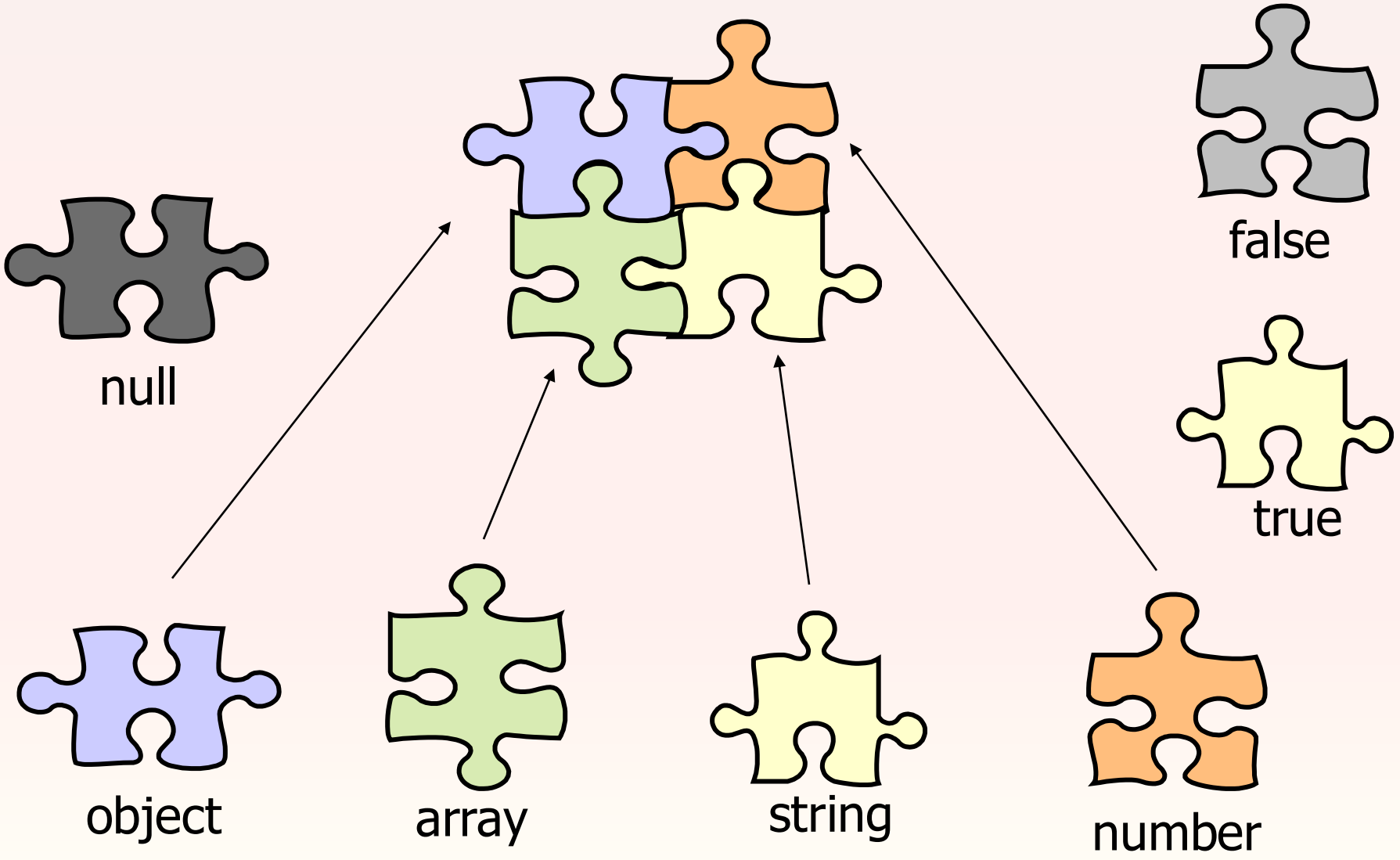
Formal specification of the language:

- https://www.rfc-editor.org/info/rfc8259

Two specifications for JSON-Schema

- https://json-schema.org/draft/2020-12/json-schema-core
- https://json-schema.org/draft/2020-12/json-schema-validation

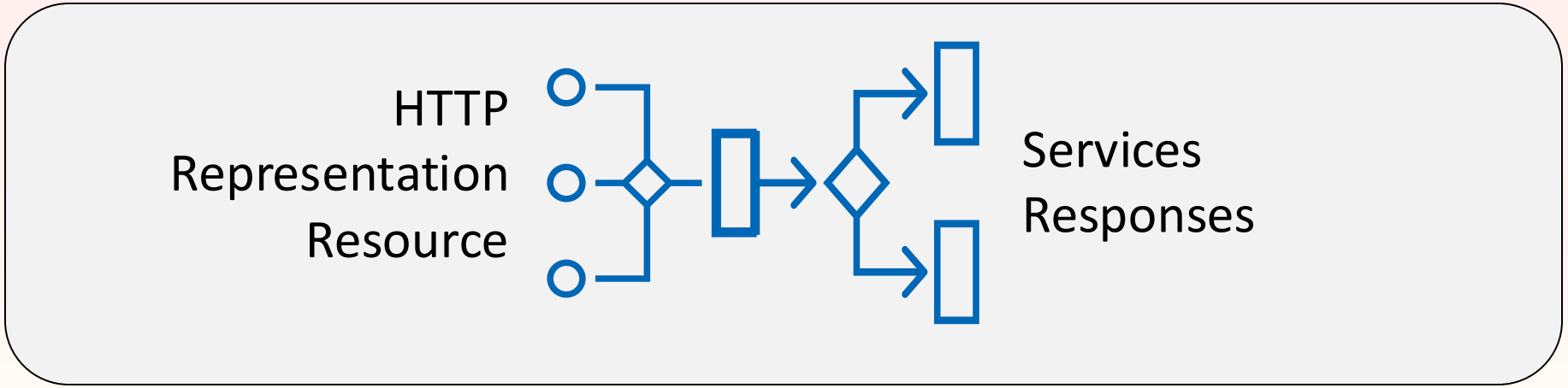
JSON: Seven Simple Components



REST: Stateless

Stateless means information necessary to handle the request is contained within the request itself.

Whether as part of the URI, query-string parameters, body, or headers.



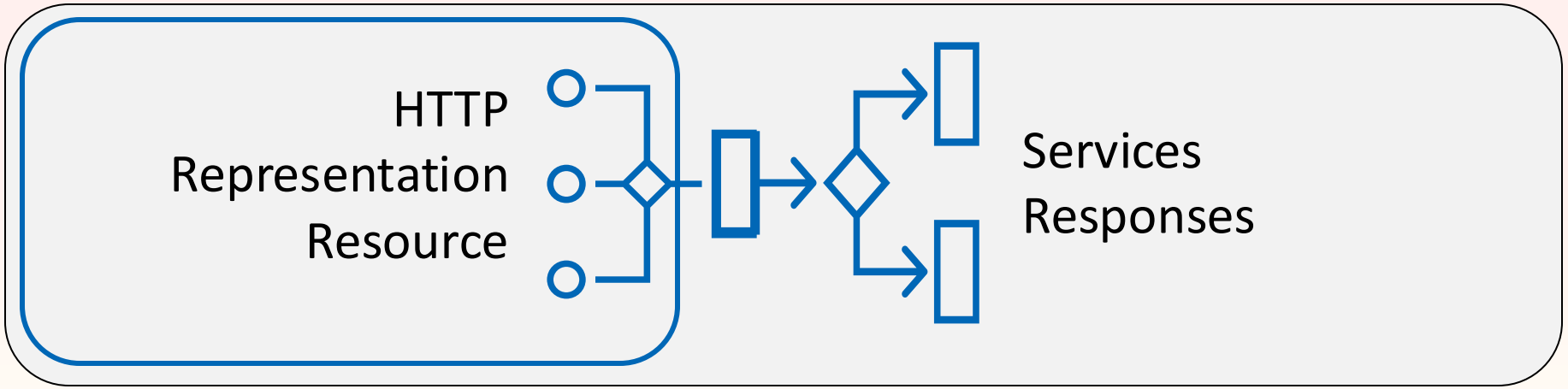
REST: Stateless (Application)

Application State:

The information needed by the server to **fulfil** a request.

- i.e necessary for the **current session** or **request**.

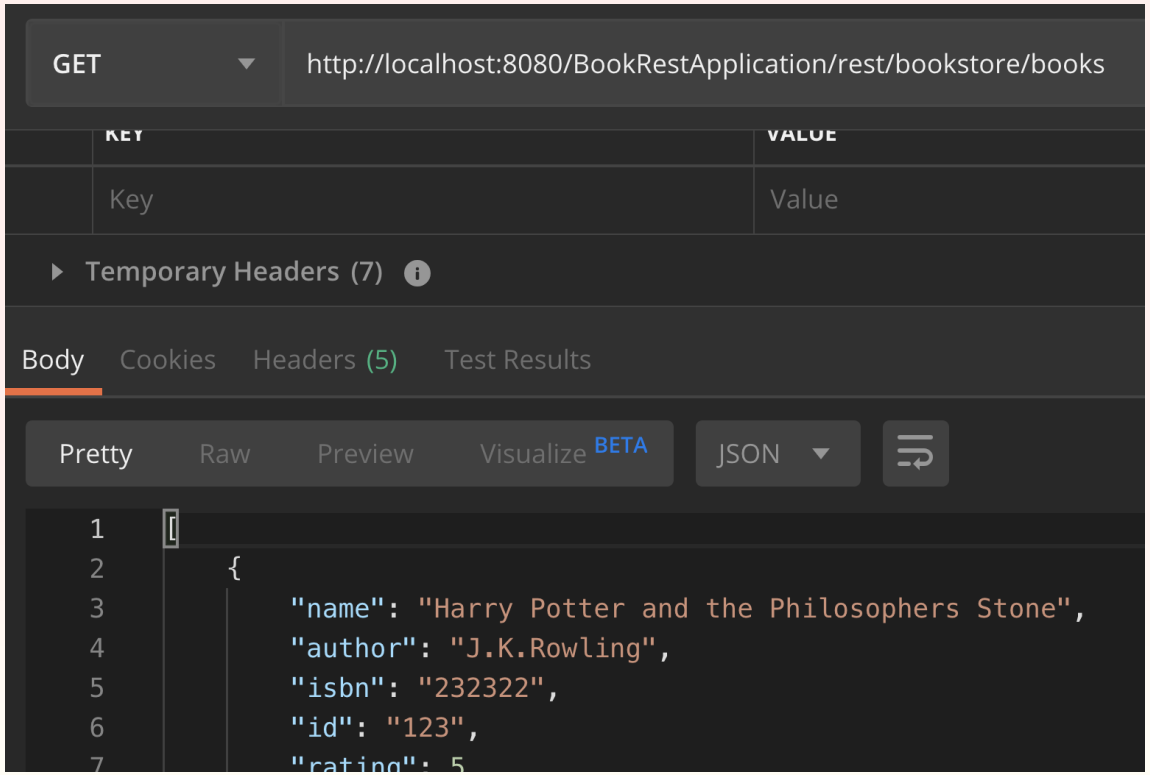
Consider application state as the data that could **vary by client**, and **per request**.



REST: Stateless Example - GET

GET on a Resource:

- Client retrieves all books from server
  (no state change happens on the application)
- JSON representation of the resource sent as response.



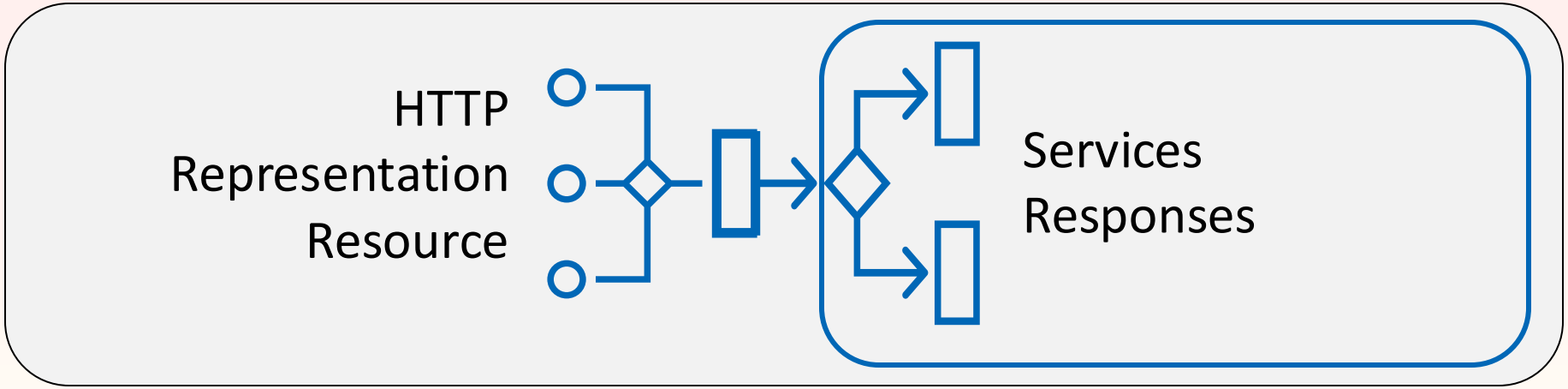
REST: Stateless (Resources)

Resource State:

The data that defines the resource representation.

- i.e the consistent responses sent across different clients.

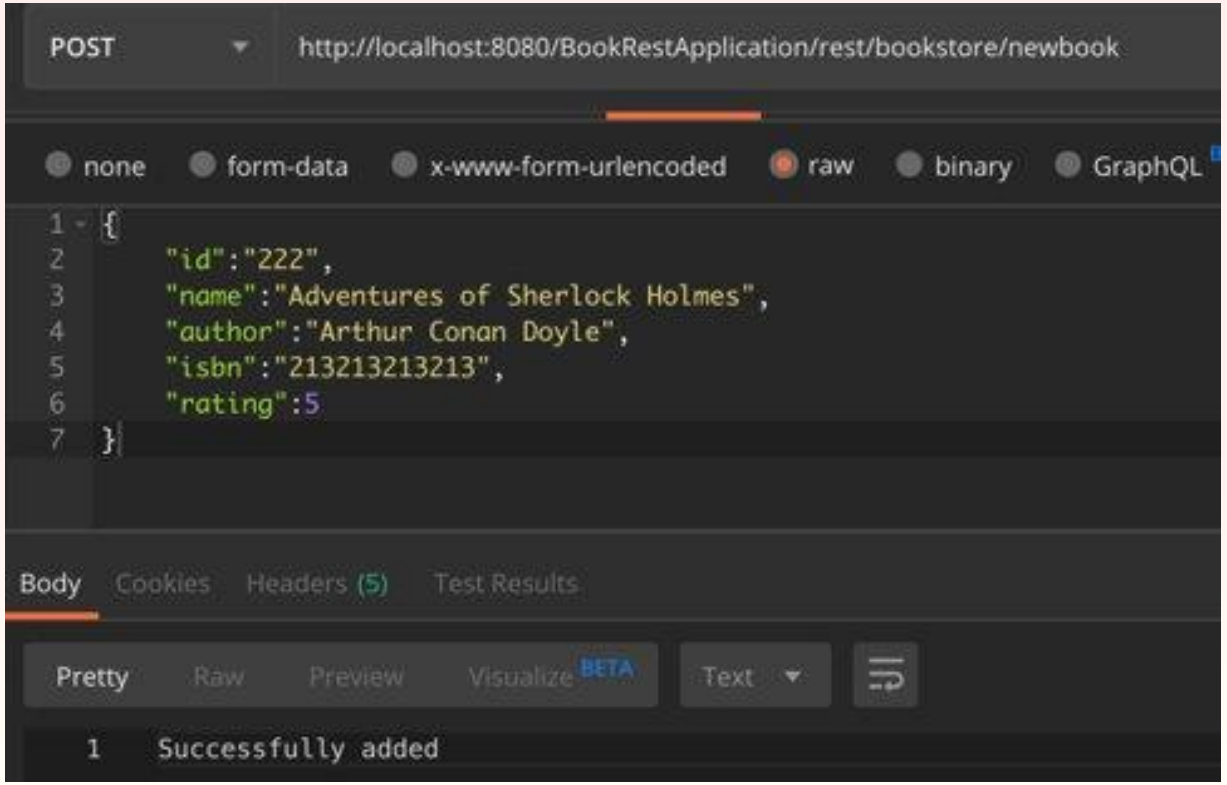
For instance, the static data stored in service database.



REST: Stateless Example - POST

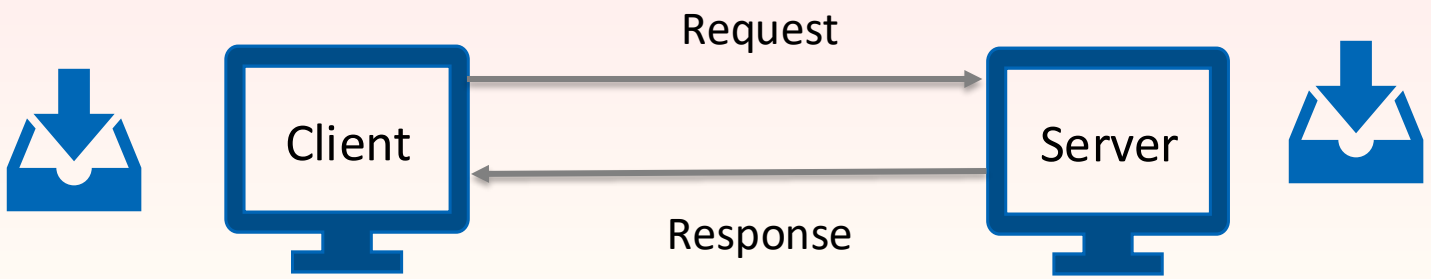
POST on a Resource:

- Client sends new book details in JSON representation.
- Tells the server to add new book to the resource thereby **changing** the **resource state**.



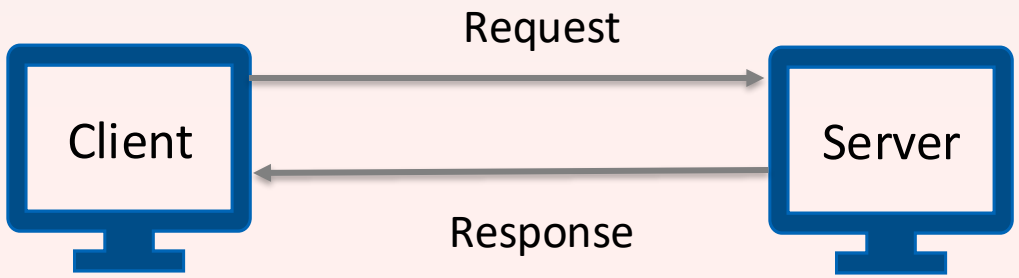
REST: Cacheable

- Clients can cache (remember) responses.
- Responses define themselves as cacheable, or not.
- Prevents clients reusing stale or inappropriate data.
- Well-managed caching (partially / completely) eliminates client–server interactions.
- Improving scalability and performance.



REST: Client-Server

Uniform interface separates clients from servers.



Not concerned with data storage.

Manages user experience

Client code more portable.

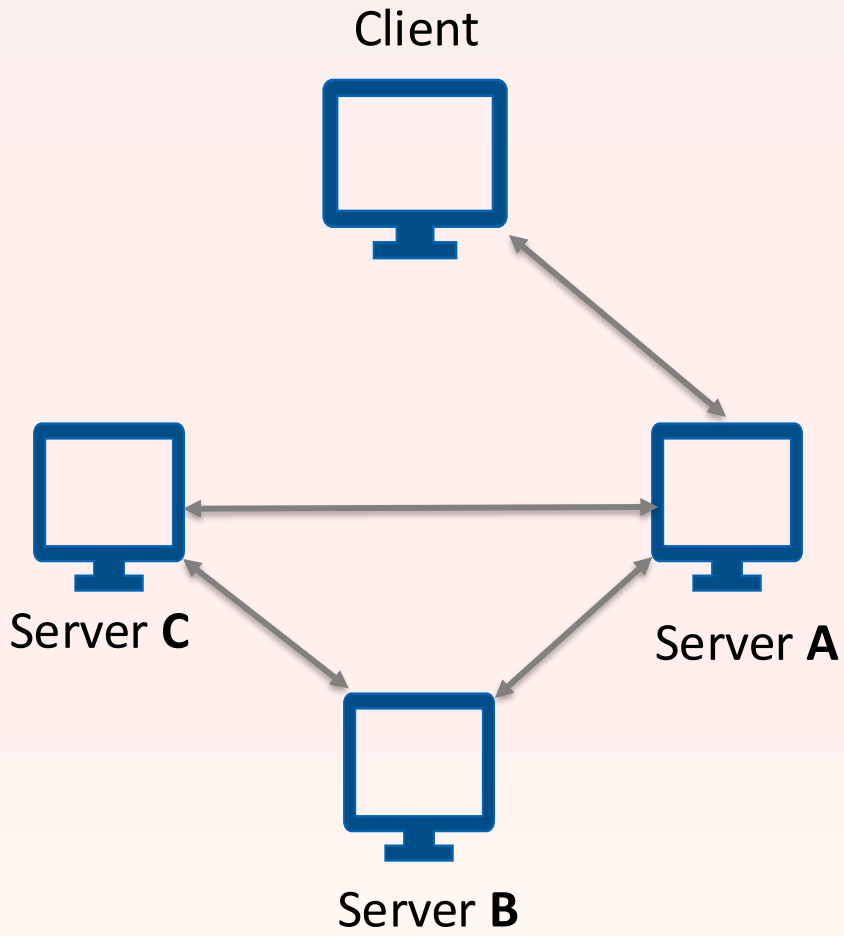
Not concerned with user interfaces or user states.

Manages resources and responding.

Server implementation simpler and more scalable.

REST: Layered System

- Client sends request to endpoint.
- Multiple servers can process a request.
- Layered system can help delegate work, contributing to servicing clients.
- Layers may also enforce security policies.



REST: Constraints

Complying Constraints ===> REST

Enables distributed hypermedia systems:

- Performance
- Scalability
- Simplicity
- Modifiability
- Visibility
- Portability
- Reliability



NOTE:
Services violating non-optional constraints are **not** strictly RESTful.

HTTP Methods: Idempotency

Idempotent Method:

Can be called several times and outcome will not change.

- GET
- HEAD
- PUT
- DELETE

are idempotent for different reasons.

In contrast, POST is not idempotent.

Calling POST multiple times can have different outcomes.

HTTP Methods: Safety

Some methods have side-effects. Safe methods do not.

Safe HTTP Methods:

- GET and HEAD (read only)

Not so safe HTTP:

- PUT usually *creates* a resource on the server.
- DELETE will *delete* the resource on the server.
- POST will *change* the resource on the server.

Representations: XML or JSON?

We can offer both:

- JSON is highly favoured and an informal convention (guess why).
- XML standards are mostly ignored (beyond syntax).
- Namespaces are not, nor should they be used in a RESTful service context.
- XML returned is more JSON like, simple and easy to read

Representations: Requests

Clients form a representation when sending requests.

Services must consider what data to support in these requests.

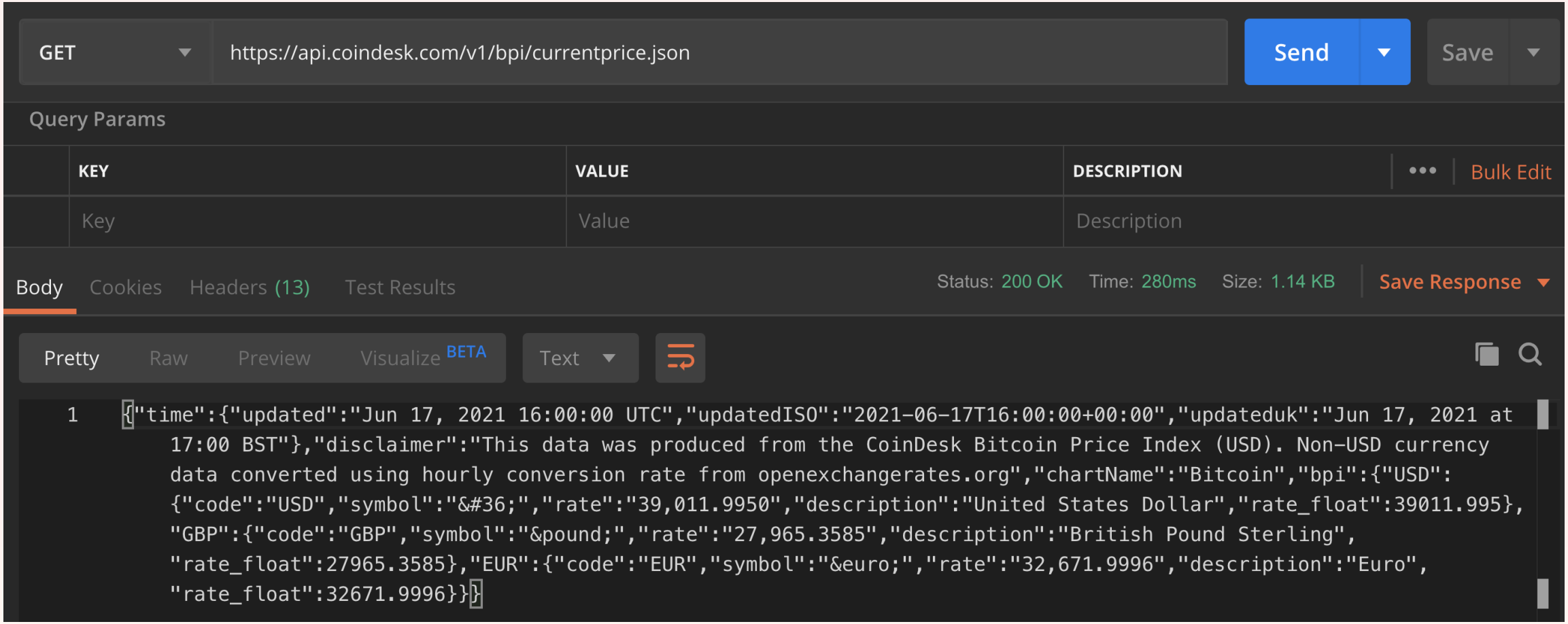
- Accept header
- File-extension-style format specifier
- Query-string parameter

etc.

```
GET /api/users/123 HTTP/1.1
Host: example.com
Accept: application/json
```

And then there's the question of naming the resources...

Representations: Example URI



Resources: Naming

A RESTful URI should refer to a resource that is a thing instead of referring to an action.

Nouns have properties. Verbs do not.

Some example resources are:

- Users of the system.
- Courses in which a student is enrolled.
- A user's timeline of posts.
- The users that follow another user.
- An article about horseback riding.

Nice article:
https://medium.com/@nadinCodeHat/rest-api-naming-conventions-and-best-practices-1c4e781eb6a5

Resources: URIs and HTTP Methods

Consider an e-commerce service:

- order system with customers, orders, line items, products…

Create a new customer in the system:

> **POST** http://www.example.com/customers

Read a customer with Customer ID# 33245:

> **GET** http://www.example.com/customers/33245

Resources: URIs and HTTP Methods

Same URI applies for other methods…

Proposed URIs for products:

`http://www.example.com/products`

Creating a new product.

> **POST** `http://www.example.com/products`

Interacting with that product:

> **GET | PUT | DELETE** `http://www.example.com/products/66432`

Question: What about creating a new order for a customer?

Resources: Choosing URIs

One option might be:

> **POST** http://www.example.com/orders



And that could work to create an order, but it's arguably outside the context of a customer.

URI that would offer better clarity:

> **POST** http://www.example.com/customers/33245/orders



Now we know we're creating an order for customer ID# 33245.

PUT vs. POST: Resource Creation

Generally: Use **POST**

PUT

Use when client decides URI for new resource.

Client has sufficient representation to generate the URI.

POST

When there's service logic responsible for generating new URI.

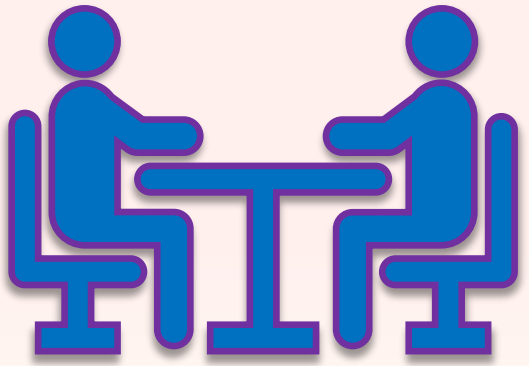
Client doesn't (or shouldn't) know what the resulting URI will be before creation.

Summary: Part 1

- JSON allows us to format service request-response representations just as well as XML
- RESTful services are defined by properties, not protocols
- With REST comes many design choices
- Design choices have best (and worse) practices

Lab: REST Design

Next lecture: Implementing REST



PART 2

REST, Java and Spring Boot

Implementing Web Services: Two approaches

There are two approaches commonly adopted for defining and implementing web services:

Contract-first:

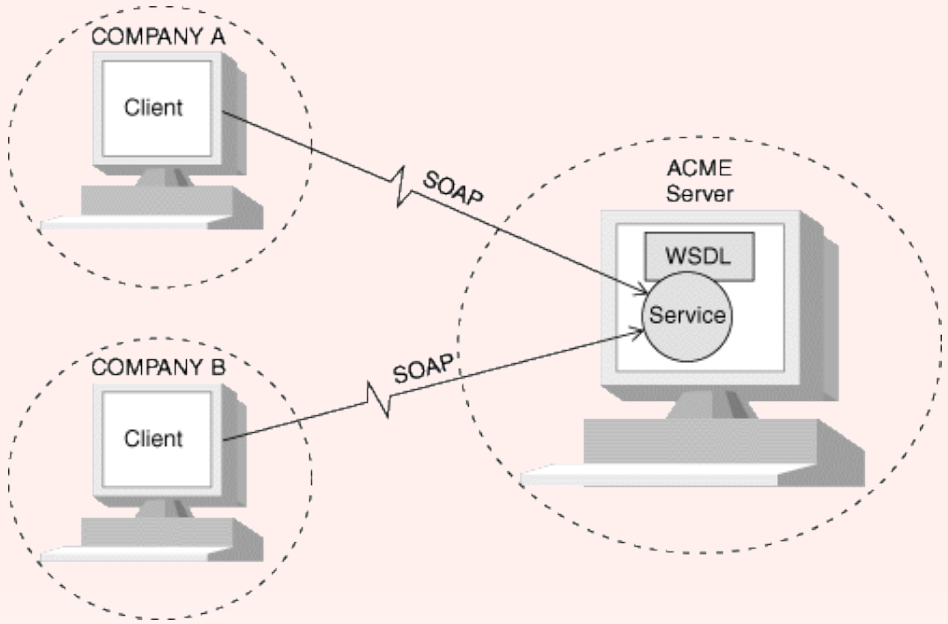
Create a contract file first and then create artifacts based on the contract agreed.

Contract-last/Code-first:

Create the Java interface first and then generate the contract from it.

Web Service Description Language: WSDL

SOAP contracts are written in a specialised version of XML called the Web Services Description Language.



It describes:

- The interface
- A set of supported operations
- Parameters for those operations
- Transmission protocol

in a standard format so clients can automatically understand how to interact with a web service.

Code First Approach: a.k.a Contract Last

Java Web Services (JWS) encourages code first by making it easy to generate this contract (i.e. the WSDL)

Once the service is published, the WSDL is generated automatically and available to clients.

Can use JDK utilities like `wsimport` and `wsgen` to generate the required client-side artifacts.

But! This means code is generated with certain assumptions in mind.

Code First Approach: Annotations

Java provides annotations (some for SOAP, others for REST)

- `@WebService`
- `@WebParam`
- `@WebResult`

etc, for programmers to determine how

- The contract is generated by the server
- Code is generated for theclient application

So, what design choices should developers start making?

REST: Resources

/students

/students/**{studentId}**

/students/{studentId}/**courses**

/students/{studentId}/courses/**{courseId}**

/students/{studentId}/courses/{courseId}/**grades**

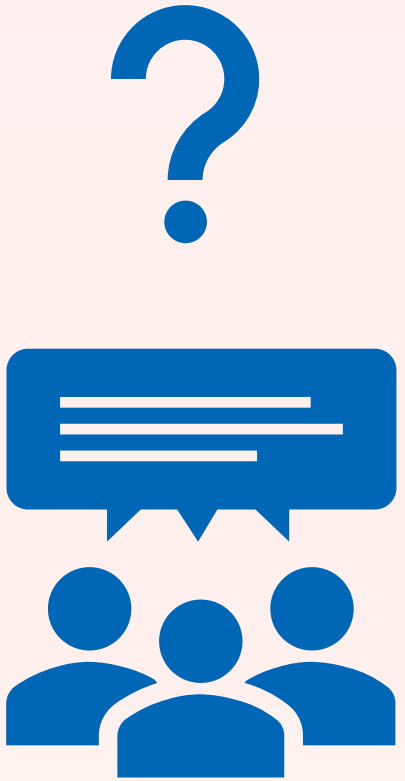
/students/**search**

/departments

/departments/**{departmentId}**

REST: Manipulating Resources

RESTful Architecture:
How do we manipulate resources?



REST: Implementation in Java

JAX-RS

- Specification for REST implementation in Java
- Reference implementation in Jersey
- Requires external container for deployment.
- A lot of manual configuration

Spring Boot

- Alternative to the (complex) Java Enterprise specification
- Uses embedded container for deployment
- Adheres to a “Convention over configuration” principle
- Larger ecosystem of modules to build enterprise systems

Spring Boot: Project Structure

```
projectName/
   |__ src/main/
   |        |__ java/
   |        |     |__ Application.java
   |        |     |__ Resource.java
   |        |__ resources/
   |              |__ application.properties
   |__ pom.xml
```

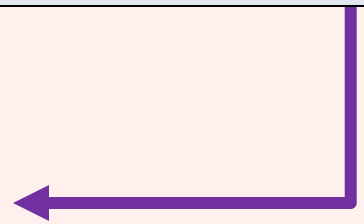
Main entry point for the service.
Marked by @SpringBootApplication



Spring Boot: Project Structure

```
projectName/
   |__ src/main/
   |        |__ java/
   |        |     |__ Application.java
   |        |     |__ Resource.java
   |        |__ resources/
   |              |__ application.properties
   |__ pom.xml
```

Definition of service endpoints and "business logic".
Marked by @RestController, @RequestMapping, and many others



Spring Boot: Project Structure

```
projectName/
   |__ src/main/
   |        |__ java/
   |        |     |__ Application.java
   |        |     |__ Resource.java
   |        |__ resources/
   |              |__ application.properties
   |__ pom.xml
```

Configuration options for the service. Custom format to spring, very simple to understand.



Spring Boot: Project Structure

```
projectName/
   |__ src/main/
   |        |__ java/
   |        |     |__ Application.java
   |        |     |__ Resource.java
   |        |__ resources/
   |              |__ application.properties
   |__ pom.xml
```

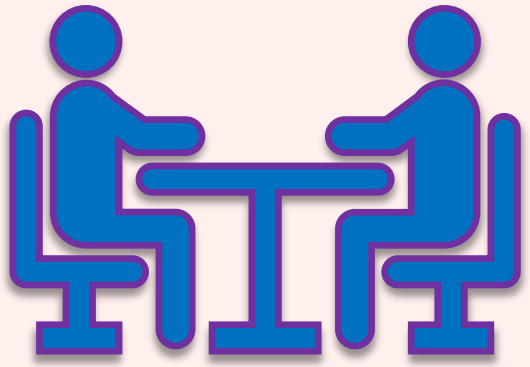
Principle Maven file. Written in XML. Defines dependencies and project information.

DEMO

Basic RESTful Web Service

Summary: Part 2

- RESTful implementations in Java are similar to the code-first approach to SOAP.
- Annotations help indicate which classes represent resources
  and which methods describe service operations
- HTTP methods can also help
  define expected means of service interaction
  and information exchange



Lab: REST Project in Spring Boot

Next lecture: Passing Parameters in REST